



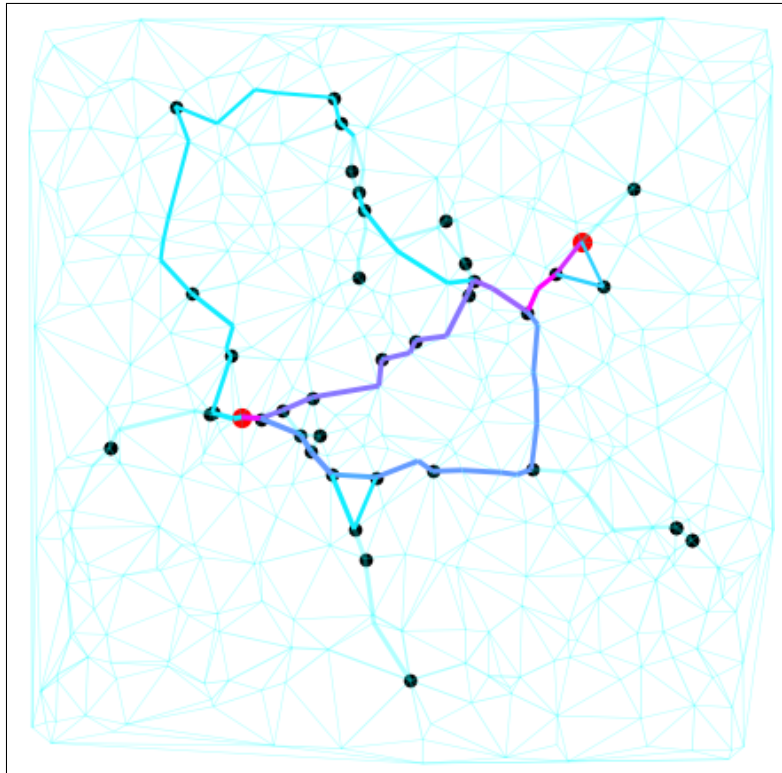
Universiteit Utrecht

Opleiding Natuur- en Sterrenkunde

Physical learning in flow networks

BACHELOR THESIS

Pieter Haas



Supervisors:

Prof. dr. MARJOLEIN DIJKSTRA

Soft Condensed Matter and Biophysics, Debye Institute for Nanomaterials Science

Prof. dr. RENÉ VAN ROIJ

Institute for Theoretical Physics,

June 13, 2025

Abstract

Rising AI power consumption projected to grow to 945 TWh by 2030 motivates looking for bio inspired alternatives. One celled Slime molds efficiently connect food sources and a slime mold inspired flow model has been used to find a network connecting the Tokyo rail network [1]. We develop Hagen-Poiseuille flow networks with local physics inspired edge adaption to achieve learning. First we use a local erosion law based on flow together with a clogging term to re create the Utrecht train network. Secondly we develop a chemical advection model to train a desired output pressure. Lastly we use the chemical advection model to achieve a 61 % accuracy on the iris dataset [2]. This demonstrate the physical flow networks are capable of learning and could inspire a physics driven learning as an energy efficient alternative for current AI .

Contents

1	Introduction	1
2	Methods	2
2.1	Flow networks	2
2.2	train network	6
2.3	Chemical advection	7
2.4	machine learning	8
3	results	9
3.1	train network	9
3.2	chemical advection	15
3.3	machine learning	19
4	Conclusions, discussion, and outlook	20
5	Laymen summary (nederlands)	21
A	Appendix	I
B	Result table	I

1 Introduction

The growth of artificial intelligence (AI) in recent years comes with an increasing problem. The estimated power consumption of AI for 2024 is 415 terawatt hours (TWh). This is about 1.5% of the global electricity consumption. The electricity consumption of AI is projected to grow towards 945 TWh by 2030, about 3% of global energy consumption [3]. A single typical query to Chat-GPT consumes 0.3 Wh of energy [4].

Nature, however, offers more efficient ways of computation. The brain uses about 20 W or 0.3 kWh per day [5] to compute complex computations. Slime moulds (*Physarum Polycephalum*) can solve complex tasks such as solving a maze by connecting 2 food sources placed at opposite sides of a maze with one tube covering the shortest possible distance [6]. And reconstructing the Tokyo rail network by adapting the width of its tubes when connecting food sources placed on the stations of a Tokyo map showing primitive intelligence. this has been reproduced numerically using a flow network based on the Hagen–Poiseuille equation and a simple erosion law based on the edge flow [1]. Slime moulds have even been shown to have a basic form of memory on the location of nutrient sources encoded in the morphology of the network-shaped. They achieve this by releasing a softening agent at the food source that is carried through the network and grows tube diameter [7].

This has led to a slime mould inspired meta-heuristic algorithm SMA (Slime Mould algorithm) that simulates the oscillation of the slime mould to find an optimal pathway connecting food sources. SMA has been used in numerous algorithms in different areas of hybridization progress, changes, and optimization [8].

In this thesis, we will explore physical learning in a flow network using a model inspired by slime moulds. We aim to uncover bio-inspired learning algorithms based on physical laws that can guide the next generation of low-power AI hardware.

We will first look into the equations governing physical flow networks based on the Hagen–Poiseuille equation and how to solve them numerically. We will then introduce a flow based erosion and clogging term and apply this model to the train stations within the Utrecht area to show that this model can create a network connecting all the train stations, similarly to what has been previously done with the Tokyo rail network by Tero *et al.* [1].

We will then introduce a chemical advection model. A chemical is released at output nodes based on difference between the output pressure and desired pressure. The chemical is carried from node to node through the network by advection. We will introduce an erosion term based on the chemical flux through the tubes and show that it can be used to train the system towards a desired (variable) output pressure at the output nodes.

2 Methods

In this chapter, we will discuss the methods that we use. We first discuss the general method for creating and solving the equations for a flow network. After a general discussion, we focus separately two methods: one for simulating a flow network that optimizes connections between train stations, and another method for solving a flow network involving the advection of a chemical.

2.1 Flow networks

To create a flow network, we first define two length scales L_x and L_y , and generate n nodes $\mathbf{r}_i$ by placing random points within the rectangular domain $[0, L_x] \times [0, L_y]$. Depending on the specific simulation, additional boundary nodes may be added. Subsequently, we construct edges between the nodes using Delaunay triangulation, which is the dual graph to the Voronoi diagram. This method creates triangles that connect each node to its neighbours while maximizing the minimum angle between all triangles. The total number of edges created using the Delaunay triangulation method is denoted by m . The length L_{ij} of the edge connecting node i and node j is defined as $L_{ij} = |\mathbf{r}_i - \mathbf{r}_j|$. For the conductance C_{ij} of the edge between node i and j , we use the Hagen–Poiseuille equation that is based on laminar flow of an incompressible Newtonian fluid through a cylindrical pipe with radius r_{ij}

$$C_{ij} = \frac{\pi r_{ij}^4}{8\mu L_{ij}}, \quad (1)$$

where μ is the dynamic viscosity. The edge radius r_{ij} is generated uniformly or randomly depending on the simulation. The flow rate through edge q_{ij} is given by the conductance C_{ij} times the pressure drop

$$q_{ij} = C_{ij}(P_i - P_j). \quad (2)$$

We solve this equation subject to Kirchhoff’s law, which requires that, for each internal node i , the incoming and outgoing flows must balance—ensuring that no flow accumulates within that node or leaks out of it

$$\sum_{i=1}^n q_{ij} = 0, \quad (3)$$

where n is the total number of edges that enter or leave node i .

Note that a flow network is analogous to an electrical network, where the flow rate q_{ij} corresponds to an electric current I_{ij} , the pressure P_i at node i to an electrical potential V_i , and the flow conductance C_{ij} to an electric conductance σ_{ij} . The current I_{ij} between node i and j follows Ohm’s law, $I_{ij} = \sigma_{ij}(V_i - V_j)$, and Kirchhoff’s current law requires that the net current at each internal node vanishes $\sum_{i=1}^n I_{ij} = 0$.

To solve these equations efficiently using numerical methods, we use linear algebra and largely follow the method of Zareei, Pan, and Amir (2022) [9]. Specifically, we represent a flow network as a directed graph G consisting of n vertices and m edges, and define the oriented incidence matrix to express this graph in matrix form. The direction of an edge is arbitrary chosen and only determines the positive direction for the fluid flow in that edge. Each row

of the $m \times n$ oriented incidence matrix corresponds to an edge and each column to a node. Thus, the number of rows of the oriented incidence matrix equals the number of vertices and number of rows is equal to the number of the edges. The (i, j) -th entry of the oriented incidence matrix is defined to be

$$\begin{cases} -1 & \text{if edge } e(i) \text{ leaves vertex } v(j) \\ +1 & \text{if edge } e(i) \text{ enters vertex } v(j) \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

This matrix compactly encodes the topology of the network and allows us to express flow conservation and constitutive relations in linear algebraic form. The vector of flow rates through all the edges $\mathbf{q}_e$ is now given by the linear algebra relation:

$$\mathbf{q}_e = \mathbf{C}_e \mathbf{D} \mathbf{P}_n, \quad (5)$$

where $\mathbf{C}_e = \text{diag}(C_1, C_2, \dots, C_m)$ is the diagonal matrix of edge conductances, $\mathbf{D}$ is the transpose of the oriented incidence matrix, and $\mathbf{P}_n$ is the vector of pressures at all the nodes. The vector of total incoming flow at each node $\mathbf{q}_n$ is given by

$$\mathbf{q}_n = \mathbf{D}^T \mathbf{C}_e \mathbf{D} \mathbf{P}_n, \quad (6)$$

which should be zero for internal nodes because of Kirchoff's law, and can only be non-zero at boundary nodes.

To obtain the flow rate through the edges $\mathbf{q}_e$ in a network described by the edge conductances $\mathbf{C}_e$ and the incidence matrix $\mathbf{D}$, we first need to determine the pressure vector $\mathbf{P}_n$ at the nodes. This is achieved by partitioning the node space into boundary and internal components. Without loss of generality, we re-partition the pressure vector $\mathbf{P}_n$ to reflect the division between boundary and internal nodes. Specifically, we write

$$\mathbf{P}_n = \begin{bmatrix} \mathbf{P}_b \\ \mathbf{P}_i \end{bmatrix}, \quad (7)$$

where $\mathbf{P}_b$ contains the pressures at the N_b boundary nodes and $\mathbf{P}_i$ contains the pressures at the remaining $N - N_b$ internal nodes. Correspondingly, we partition the incidence matrix $\mathbf{D}$ into

$$\mathbf{D} = [\mathbf{D}_b \ \mathbf{D}_i], \quad (8)$$

where $\mathbf{D}_b$ represents the submatrix of $\mathbf{D}$ consisting of the first N_b columns, corresponding to the boundary nodes, and $\mathbf{D}_i$ denotes the submatrix consisting of the remaining $N - N_b$ columns, corresponding to the internal nodes. Similarly, we re-partition Eq. 5 using the block matrix representation in equation 9 adapted from Zareei, Pan and Amir (2022) [9]

$$\left[\begin{array}{c|c} \mathbf{D}_b^\top \mathbf{C} \mathbf{D}_b & \mathbf{D}_b^\top \mathbf{C} \mathbf{D}_i \\ \hline \mathbf{D}_i^\top \mathbf{C} \mathbf{D}_b & \mathbf{D}_i^\top \mathbf{C} \mathbf{D}_i \end{array} \right] \begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_{N_B} \\ \hline P_{N_B+1} \\ \vdots \\ P_N \end{bmatrix} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_{N_B} \\ \hline 0 \\ \vdots \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} \mathbf{A}_{bb} & \mathbf{A}_{bi} \\ \mathbf{A}_{ib} & \mathbf{A}_{ii} \end{bmatrix} \begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_{N_B} \\ \hline P_{N_B+1} \\ \vdots \\ P_N \end{bmatrix} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_{N_B} \\ \hline 0 \\ \vdots \\ 0 \end{bmatrix}, \quad (9)$$

where we have defined the auxiliary matrices $\mathbf{A}_{st} = \mathbf{D}_s^\top \mathbf{C} \mathbf{D}_t$ for $s, t \in \{b, i\}$. Equations 2 and 3 describing the flow network is now reduced to solving the linear system

$$\mathbf{A}_{bb} \mathbf{P}_b + \mathbf{A}_{bi} \mathbf{P}_i = \mathbf{q}_b, \quad (10a)$$

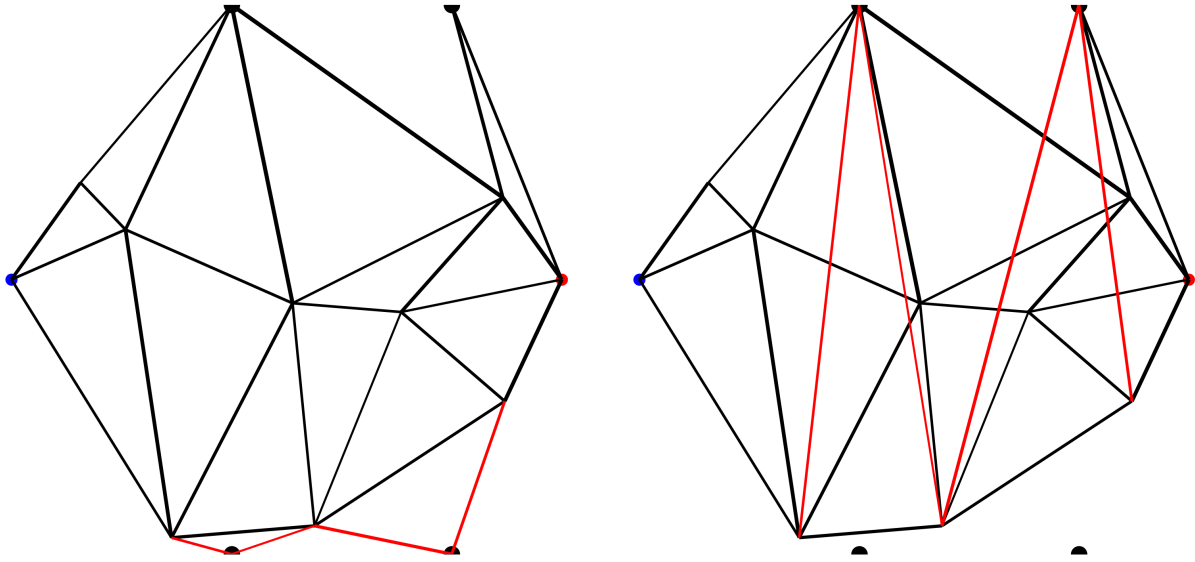
$$\mathbf{A}_{ib} \mathbf{P}_b + \mathbf{A}_{ii} \mathbf{P}_i = 0. \quad (10b)$$

If the flow rate vector $\mathbf{q}_b$ at the boundary nodes is given, the system of linear equations 10a and 10b need to be solved simultaneously to find the pressure vector $\mathbf{P}_b$ at the boundary nodes and the pressure vector $\mathbf{P}_i$ at the internal nodes. However, if the pressure vector at the boundary nodes is given, only Eq. 10b needs to be solved, and then Eq. 10a can be used to find the required flow rates $\mathbf{q}_b$ at the boundary nodes.

Solving for $\mathbf{P}_i$ using Eq. 10b, as in $\mathbf{P}_i = (\mathbf{A}_{ii})^{-1} (-\mathbf{A}_{ib} \mathbf{P}_b)$, can lead to numerical instability because $\mathbf{A}_{ii}$ is not always invertible. We therefore use the Moore–Penrose inverse (pseudo inverse) of $\mathbf{A}_{ii}$, denoted by $\mathbf{A}_{ii}^+$. Specifically, we compute the pseudo inverse using the singular-value decomposition (SVD) of $\mathbf{A}_{ii}$, given by $\mathbf{A}_{ii} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$. The pseudo inverse is then $\mathbf{A}_{ii}^+ = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^T$, where $\mathbf{\Sigma}^+$ is the inverse of the non-zero elements of $\mathbf{\Sigma}$. This allows us to compute the internal pressures using $\mathbf{P}_i = (\mathbf{A}_{ii})^+ (-\mathbf{A}_{ib} \mathbf{P}_b)$, resulting in a more numerically stable solution.

To improve runtime we use the Cholesky decomposition of $\mathbf{A}_{ii}$, where $\mathbf{A}_{ii}$ is decomposed into the product of a lower triangular matrix and its transpose $\mathbf{A}_{ii} = \mathbf{L} \mathbf{L}^T$. This approach requires that $\mathbf{A}_{ii}$ is a Hermitian, positive-definite matrix. When this condition is not met, we instead use the pseudo inverse as a fallback to ensure a stable solution. A similar method can be used to solve for the system of equations in Eq. 10 for the case of a boundary flux, using the full block matrix $\mathbf{A}$ instead of just $\mathbf{A}_{ii}$.

If we want to make the network vertically periodic we run into the problem that it is not trivial which bottom nodes should be connected to which top node. To solve this problem we add nodes at the top and 'ghost' nodes at the bottom vertically aligned on the same x position. We use these 'ghost' nodes to create the edges using Delaunay triangulation, but they are not part of the final network. To make the network periodic we redirect the edges connected to the bottom 'ghost' nodes to the top nodes. We keep the original edge length as if they are connected to the bottom node. For visualisation we keep the ghost nodes and edges to the ghost nodes. We cut the top and bottom nodes in half to show that they are the same node i.e. flow going in to a 'ghost' node at the bottom the flow comes out at the top node. To prevent the added top and bottom nodes from influencing the network, we remove the edges connecting the top nodes to each other, as well as those among the bottom nodes.



(a) Network that shows the ghost nodes and original edges of the ghost nodes in red (b) network that shows that the redirected edges towards the top nodes in red and shows that there are no edges towards the 'ghost' nodes

Figure 1: Comparison of (b) the raw periodic flow network and (a) the shared-node visualisation. That shows that nodes near the bottom are actually connected to the top nodes in red edges but are visualized with shared nodes to prevent edges crossing the entire network.

2.2 train network

To investigate the Utrecht train network using our physical flow network, we largely follow the method from the work: Rules for biologically inspired adaptive network design (Tero et al., 2010, in Science) [1]. We first create a network of size $Nx = Ny = 20$. for the location of the Utrecht train stations we use the NS API [10] that provides the longitude and latitude of all the NS train stations. We select the Utrecht train stations by selecting the right latitude and longitude of $51.90 < \text{latitude} < 52.29$ and $4.865 < \text{longitude} < 5.56$. We linearly scale the x and y positions of the train stations between 3 and 17 while neglecting the earths curvature. This places the stations inside our grid and leave space to the boundary to minimise the effect of the boundaries. We set the edge width of every edge to 1. Every time step we randomly select 2 station nodes as boundary nodes and find the internal nodes as the difference of all nodes to the boundary nodes. We calculate the $\mathbf{D}$, $\mathbf{D}_b$, $\mathbf{D}_n$ matrices and the $\mathbf{C}$ matrix. We use them to calculate the $\mathbf{A}$ matrices and use these matrices to solve for $\mathbf{q}$ and $\mathbf{P}$ as described in 2.1. We use a boundary flux of $\mathbf{q}_{BC} = \begin{bmatrix} q_0 \\ -q_0 \end{bmatrix}$, where $q_0 = 2$: we impose an incoming flux of 2 on one station node and an equal outgoing flux on the second station node. Because we only use the absolute value of flow for erosion it doesn't matter which station has an incoming or outgoing flux. We use the erosion law of equation 11.

$$\frac{dD_{ij}}{dt} = \frac{|q_{ij}|^\gamma}{1 + |q_{ij}|^\gamma} - \beta D_{ij} \quad (11)$$

Here $|q_{ij}|$ is the absolute value of the flow. γ is a constant that governs the erosion that we set to 1.8. In the paper by Tero et al[1], this resulted in a network both similar to the rail network and the slime mould. The constant β is set to 0.05 and determines the amount of clogging that makes edges disappear. $D_{ij} = C_{ij} \cdot L_{ij}$, we take L_{ij} out of the conductivity because it is constant during the simulation.

$$D_{ij} = \frac{\pi r_{ij}^4}{8\mu} \quad (12)$$

We set dt to 1 and update the edge widths according to equation 12. We repeat this process for 500 steps. After 500 steps we find our train network by selecting the edge of a width larger than 1 that grew during the simulation.

To scale the boundary flux of the station nodes we use the public NS data [11], where we have merged the data set of the number of passengers that enter a train with the data set of the position of the train stations, this dataset is available in the supplementary material A. For the boundary flux q_0 we scale the passenger numbers between 1.5 and 2.5. Each time step, we pick the station with the least amount of passengers as our boundary flux.

The full code is also available in the appendix A

2.3 Chemical advection

To investigate the chemical advection model we create a network of size $N_x = N_y = 12$. We add two output nodes at $x = 0.5N_x$ and $y = \frac{1}{4}N_y$ and $\frac{3}{4}N_y$. The boundary nodes are the most right and most left nodes of the network and have a boundary pressure $\mathbf{P}_b = (0 \ 10)^T$. For the edge widths we use a random uniform distribution between 5 and 14. For the release of the chemical we follow Appendix A from Anisetti, Scellier and Schwarz (2023, ASP Physical Review Research) [12]. For each release of the chemical we compare the pressure at the output node $P(a)$ with a desired output pressure $P_d(a)$ which can be constant or variable. We release a chemical s_+ or s_- according to formula 13.

$$\begin{cases} s_-(a) = \lambda(P_d(a) - P(a)), & P(a) < P_d(a) \\ s_+(a) = \lambda(P(a) - P_d(a)), & P(a) > P_d(a) \end{cases} \quad (13)$$

Here λ is a constant that governs the amount of chemical that is released that we set to 100. If the output pressure is higher than the desired output pressure s_+ is released and if it's lower s_- is released. These chemicals are then carried through the network by advection. Let $\mathcal{N}(a)$ be the neighbouring nodes of node a . For all nodes $b \in \mathcal{N}(a)$ the chemical in node a at step t is distributed according to equation 14

$$s_{+/-}(b, t+1) = s_{+/-}(a, t) \times \frac{q_{ab}}{\sum_{x \in \mathcal{N}(a)} q_{ax}} \quad (14)$$

The chemical is distributed to its neighbouring nodes according to the fraction of its outgoing flow that goes to that node. This chemical distribution is an approximation by not taking diffusion into account and only advection directly from node to node every time step.

The edges are then eroded according to equation 15.

$$\frac{dC_{ij}}{dt} = \zeta(S_+ - S_-) \quad (15)$$

Here ζ is a constant that we set to 50 that governs the strength of the erosion. S_+ and S_- are the flux of the plus and minus chemical through the edge.

Every time step during the simulation the chemical is distributed according to equation 14 and the edge widths are updated according to equation 15. The matrices $\mathbf{D}$, $\mathbf{C}$ and $\mathbf{A}$ are calculated. Using these matrices $\mathbf{P}$ and $\mathbf{q}$ are calculated. The total amount of chemical in the system is calculated when it is 0 and all of the chemical is drained out of the system the chemical is released according to equation 13. A shortcoming of this advection model is that the chemical only reaches nodes and edges downstream of the output nodes. This means that only edges downstream are eroded. We therefore flip the source and the sink node every time that a chemical is relapsed by multiplying the boundary pressure by -1 and multiplying the desired pressure by -1 as well; $\mathbf{P}_b = -\mathbf{P}_b$ and $\mathbf{P}_d = -\mathbf{P}_d$. This makes sure that the chemical erodes edges on both the left and the right of the output nodes. We flip the desired pressure as well to make sure that it doesn't interfering with the learning.

The full code is available in the appendix A

2.4 machine learning

To test whether our chemical advection model is capable of machine learning we test on the iris dataset of the UC Irvine Machine Learning Repository [2]. This is a dataset that is commonly used to test models and has 4 inputs that represent characteristic of iris plants : sepal length, sepal width, petal length and petal width as output types of iris plant: Iris Setosa, Iris Versicolour, and Iris Virginica. The dataset has 150 entries equally distributed among the 3 types of iris. The aim is to identify the right type of iris based on the characteristic. Baseline model performances that are publicly available typically have a accuracy of above 90 % for this dataset.

We generate a network of size $N_x = N_y = 12$ of $0 < y < N_y$ but $1 < y < N_x - 1$. We generate 4 input nodes on the left and right side of the network at $x = 0$ and $x = N_x$ with y coordinates of $\frac{1}{5}N_y$, $\frac{2}{5}N_y$, $\frac{3}{5}N_y$ and $\frac{4}{5}N_y$ to create a pressure drop and flow from left to right. we generate 3 output nodes at $x = \frac{1}{2}N_x$ and y is $\frac{2}{10}N_y$, $\frac{5}{10}N_y$ and $\frac{8}{10}N_y$ in the middle of the network. We normalise the data of the iris dataset between 0 and 4 for each category. We add 5 to ensure that the pressure drop between the sink nodes and source nodes is always greater than the pressure drop among source nodes. We use a boundary pressure of the scaled data on one side and 0 on the other side. For each category of the 4 iris characteristics the 50 entries are split into 25 training and 25 testing entries which are then shuffled. During the training phase a boundary pressure $\mathbf{P}_{\mathbf{BC}}$ is selected from the training data. The desired output pressure $\mathbf{P}_{\mathbf{d}}$ is the average of the boundary nodes where the desired pressure of the target node is increases by 2Δ and the other desired pressures are lowered by Δ . This nudges the pressure up for the right output nodes. We use a value of 0.1 for Δ . The simulation for each boundary pressure is then run for 750 steps according to the method of section 2.3. During the testing phase a boundary pressure is selected from the testing data and the output node with the highest pressure is selected as the predicted output iris.

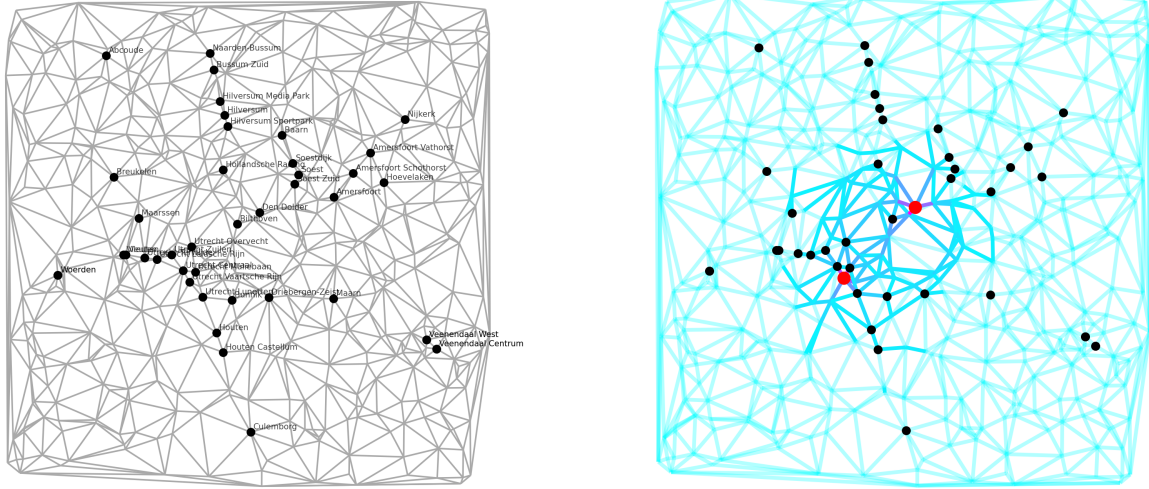
The full code is available in the appendix A

3 results

3.1 train network

in this section we present the results of re creating the Utrecht train network using our physical flow network as described in the method in 2.2

We start of with the network of figure 2a and flow of figure 2b:



(a) Starting network where the grey lines represent edges and the black dots are the train stations with their corresponding name.

(b) Starting flow network where the black dots represent train stations treated as internal nodes and the red dots are boundary stations that have an incoming or outgoing boundary flux. Edge color is a cool color map ranging from light blue to pink from least to most flow.

Figure 2: Start network and start flow

Initially, the flow does not follow a single, shortest path route but instead, it spreads diffusely through many edges connecting the boundary nodes. However as the simulation proceeds the erosion law of equation 11 causes some edges to widen because of the first term, but most edges contract because of the second term. This causes the flow to concentrate in a smaller set of edges. This process converges to a stable network of edges connecting all the stations. The final flow can be seen in figure 3.

An animation of this process together with the code that is used for this simulation is made available in the appendix A

figure 3 shows the flow of the final time step. We see that the flow flows along a network connection all stations.

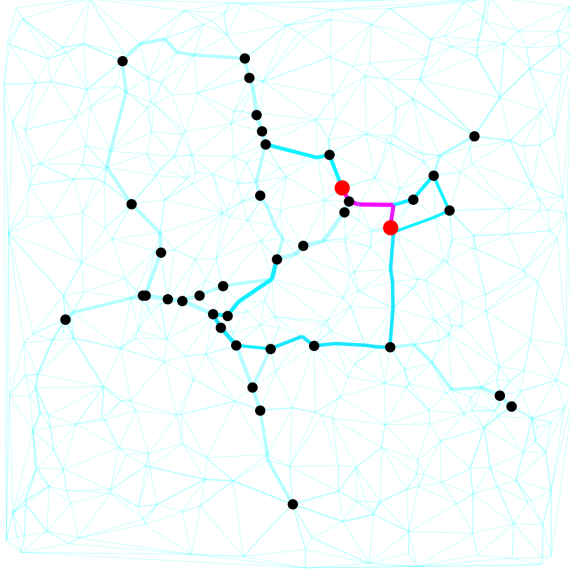


Figure 3: final flow where the flow flows from between the station nodes indicated in red through the network of edges connecting the stations indicated in black

We select the edges wider than the original edge width of 1 to select the final network 4 and see that it connects all stations with a network.

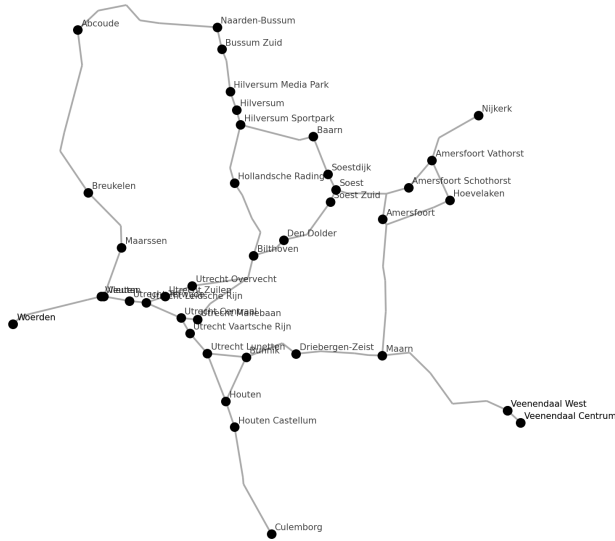
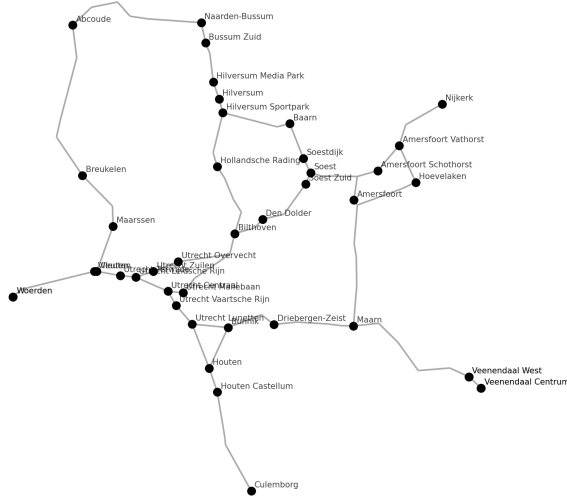
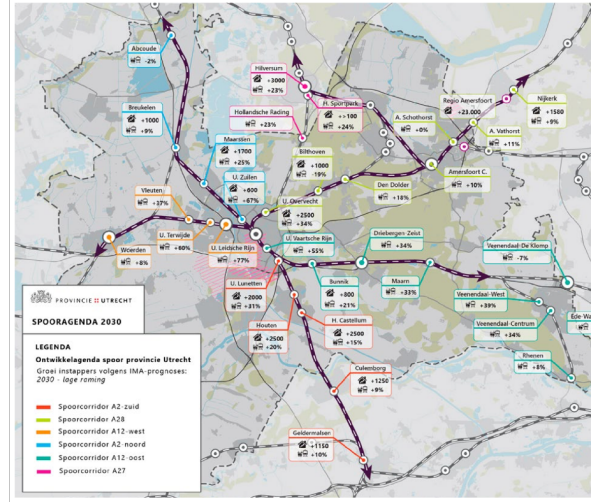


Figure 4: Final network of $\gamma = 1.8$ and $q_0 = 0.2$ where the edge wider than 1 are selected.

If we compare the final network 5a with the actual train network 5b taken from a report of the province Utrecht [13] We notice that there are a large number of similarities but also some key differences. Our network seems to have more connections than the actual network most notably between Maarn and Amersfoort and between Houten en Bunnik. Our simulated network has a junction outside of the train stations between Soest Zuid en Amersfoort that is abesnt in the actual network. But our network is also missing some connections that are present in the actual network such as between Baarn and Amersfoort and between Breukelen and Utrecht Centraal.



(a) final network fo the simulation

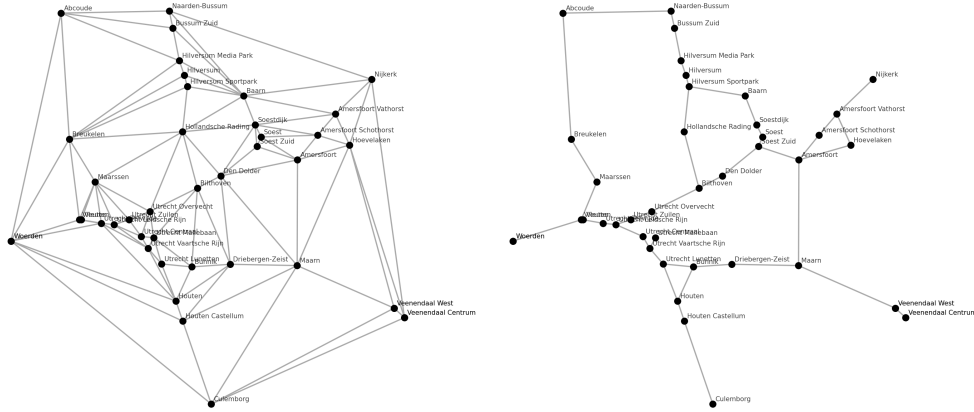


(b) actual train network in the province of Utrecht

Figure 5: comparisons of the final network of the simulation and the actual train network

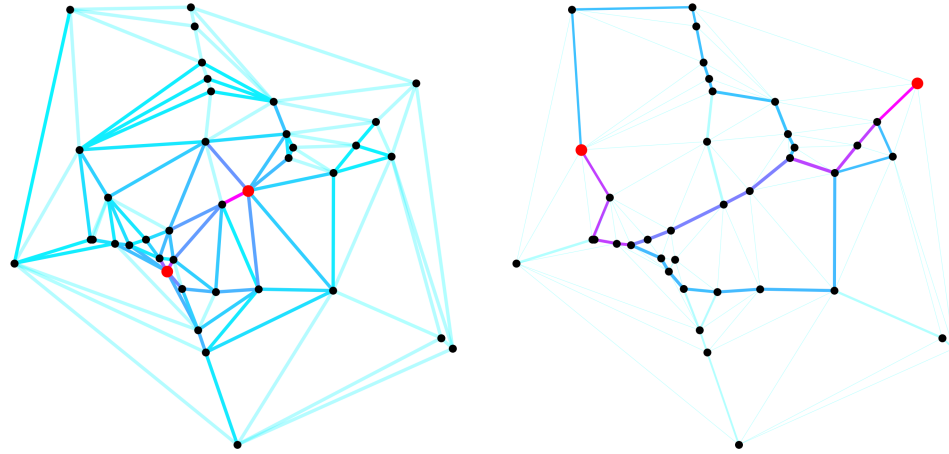
Changing the value of the γ and β and q_0 parameters changes the shape of the final network. if we change γ to 1.15 , q_0 to 0.2 β to 0.003 we get the network of figure 6. In the paper of Tero et al [1] this resulted in a network with similar topology to the Tokyo rail network. We notice that these parameters lead to a network with more connections. For example between Culemborg and Veenendaal centrum and between Veenendaal centrum and Hoevelaken on the edge of the network but also between Maarsen and Hollandsche Rading in the centre of the network. For the Utrecht train network this leads to a network that is less similar to the actual train network compared to the previous settings of the parameters.

Our new starting network in fig7a has no additional nodes besides the station nodes.



(a) Start network of non embedded train stations where only the stations are connected using Delaunay triangulation
(b) Final network of selected edges for stations where only the stations are connected using Delaunay triangulation

Figure 7



(a) Starting flow for non embedded stations with $\gamma = 1.8$ and $q_0 = 2$
(b) Final flow for non embedded stations with $\gamma = 1.8$ and $q_0 = 2$

Figure 8

We again notice in figure 8a that at the start the flow from the source to the sink nodes goes through many edges. Through erosion, a stable network forms 8b. An animation of this process is available in the appendixA We see that this network 7b has fewer connections than the embedded network 4. An effect of not embedding means that there are no junctions outside of the train stations.

We also looked at this network with the parameters $\gamma = 1.15$ and $q_0 = 0.2$ we again see that this leads to a network where the flow connects all stations 9. An animation of this process is available in the appendix A.

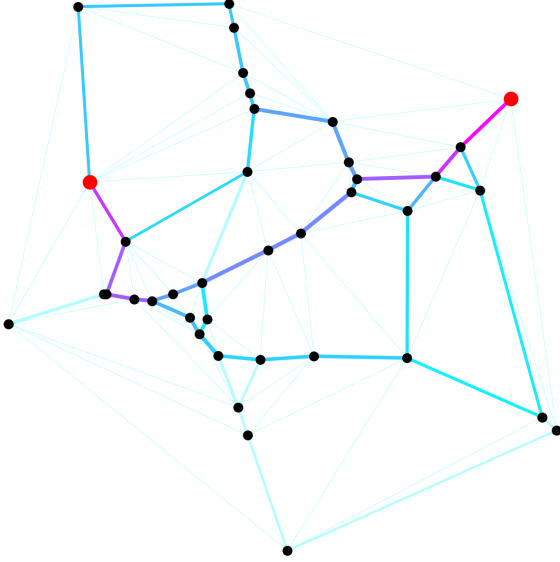


Figure 9: Final flow for non embedded stations with $\gamma = 1.15$ and $q_0 = 0.2$

This leads to a final network of figure 10 with more connections than the parameters of $\gamma = 1.8$ and $q_0 = 2$ of figure 7b . We again see that the network is similar to the embedded network of figure 6b .

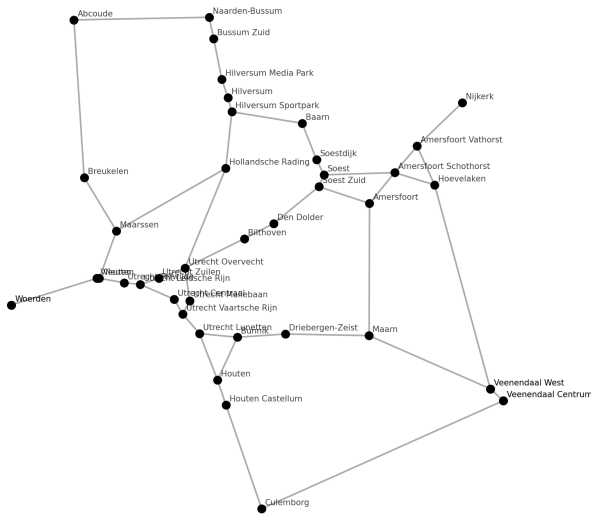


Figure 10: Final network of selected edges for $\gamma = 1.15$ and $q_0 = 0.2$

3.2 chemical advection

in this section we present the results of using a chemical advection model to train the network to give a desired output pressure at 2 output nodes as described in the method in 2.3

We start of with the flow network of figure 11. It is a square network with a pressure drop from left to right and two output nodes in the middle.

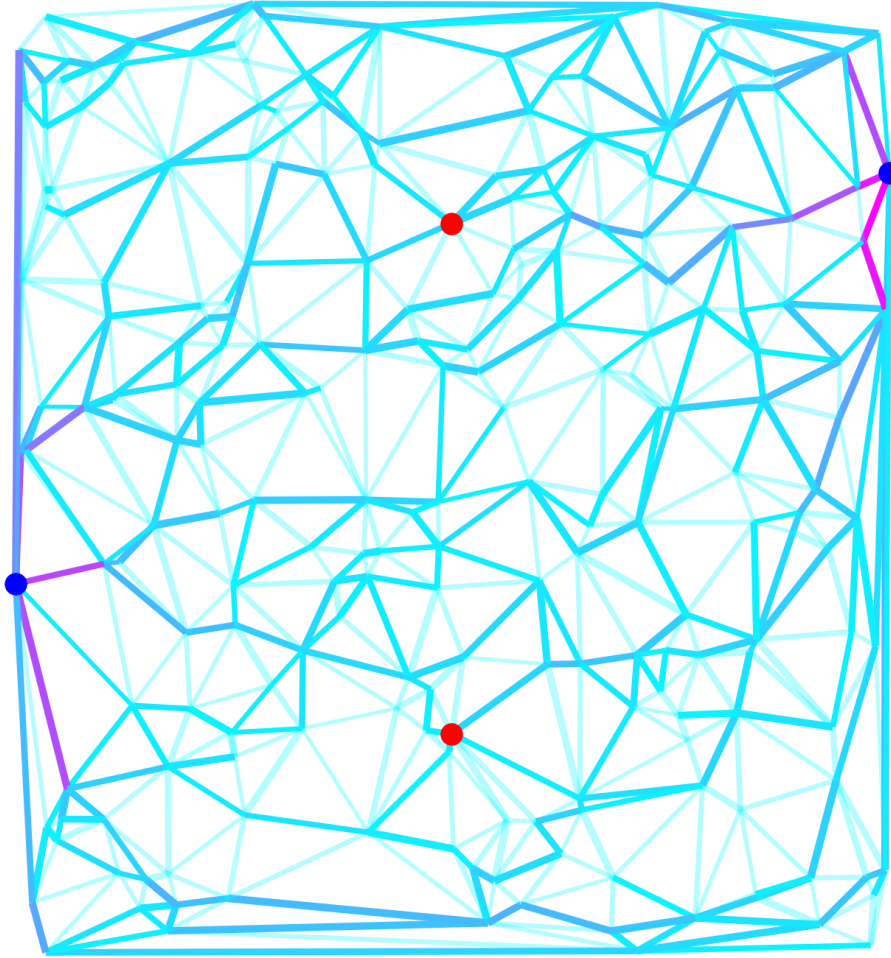


Figure 11: starting network of size $N_x = N_y = 12$, connected with Delaunay triangularization, 2 added output nodes at $x = 6$ and $y = 3, 9$ indicated in red and a boundary pressure $\mathbf{P}_b = (0 \ 10)^T$ on nodes indicated in blue

We first look at a constant desired output pressure of $\mathbf{P}_d = (3 \ 7)^T$. In figure 12 we see that the output pressure of both nodes that are in the middle start of with a pressure around 5 which is halfway between 0 and 10. Error is defined as the absolute difference between the output pressure and the desired pressure $|P_d(a) - P(a)|$. This means that the error for both output nodes is around 2 at the start. We see in figure 12 that for both output nodes the output pressure reaches the desired output pressure within a 1000 steps.

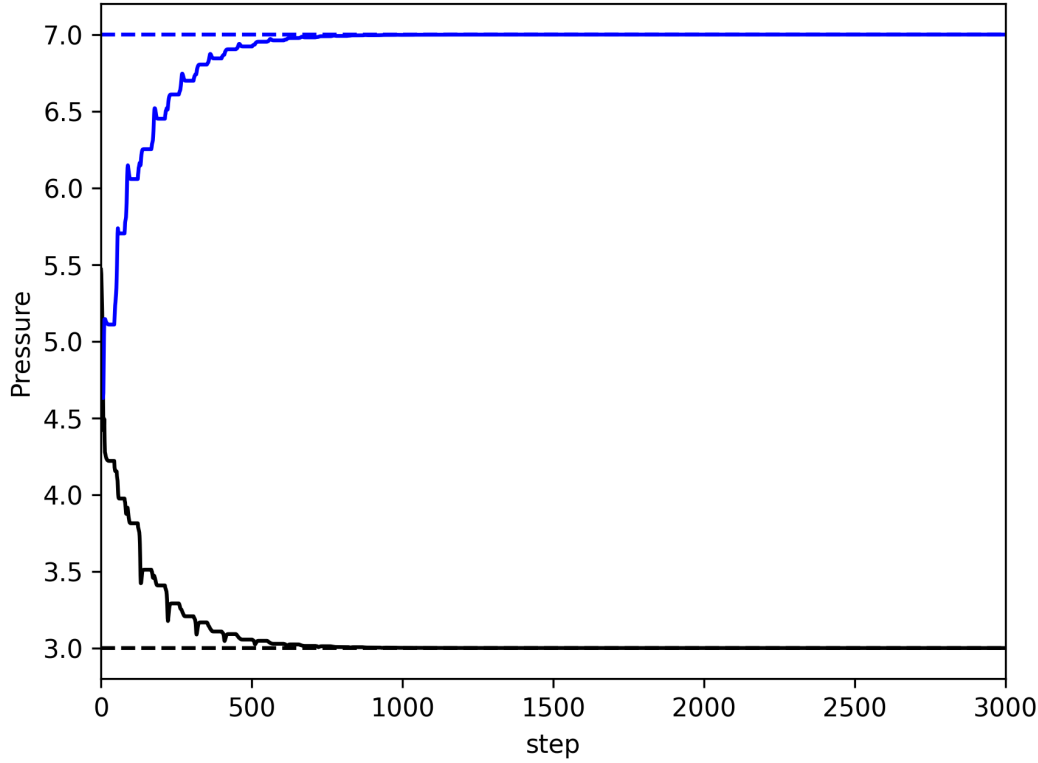


Figure 12: Pressure for 2 output nodes indicated in solid blue and solid black together with the desired pressure $\mathbf{P_d} = (3 \ 7)^T$ indicated with a dashed line of the same colours.

In figure 13a we see that the error reaches zero within a 1000 steps. In figure 13b we see that after a 1000 steps the error reaches order of magnitude of 10^{-3} but that the error continues to decrease exponentially to an error of 10^{-8} after 3000 steps.

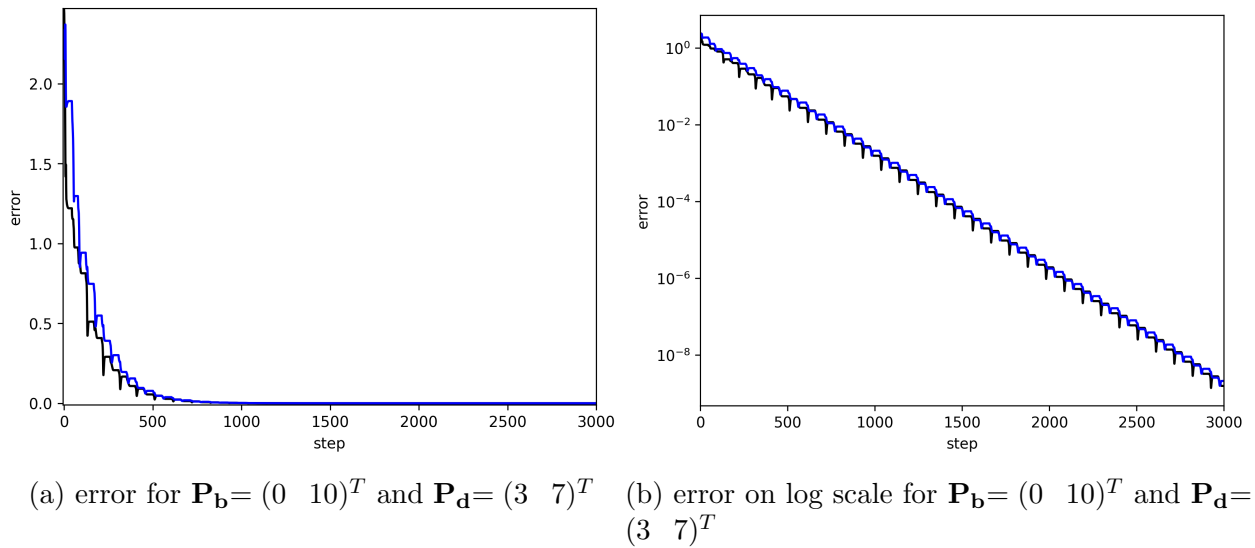


Figure 13: error for constant pressure

We use a linear relation for the desired output pressure of $\mathbf{P_d} = \mathbf{a} + \mathbf{b} \cdot \text{steps}$. For $\mathbf{a}$ we use starting output pressure at the output nodes. For $\mathbf{b}$ we use $1 \frac{2}{\text{max steps}}$. such that the desired pressure increases by 2 during the length of the simulation. we have set the max steps to 3000. In figure 14 we see that the output pressure is able to follow the linear increasing desired pressure with a stair case like output pressure.

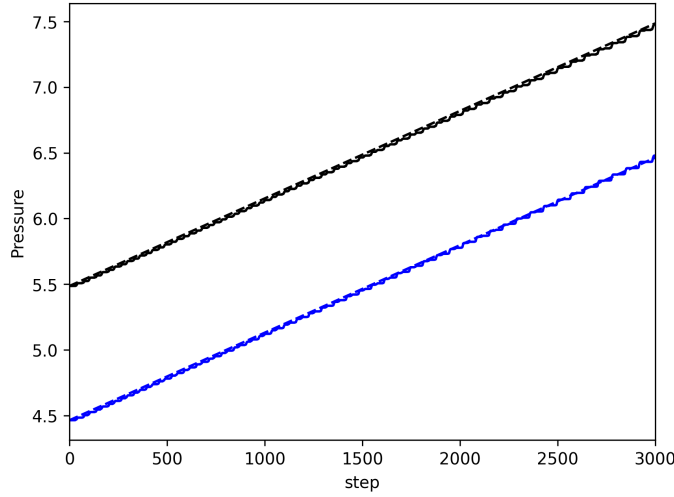


Figure 14: output pressure in solid line and desired pressure in dashed line for the two output nodes in black and blue, for a linear desired pressure

For the sinusoidal desired pressure we use: $\mathbf{P_d} = \mathbf{a} + \mathbf{b} \cdot \sin(\omega \cdot \text{steps})$. We used an $\omega = \frac{2\pi}{\text{max steps}}$, $\mathbf{a}$ of the starting output pressure and $\mathbf{b} = 1 \cdot 2$ such that the desired pressure is a sinus with amplitude 2 and one full phase during the simulation. In figure 15 we see that the output pressure is again able to follow the desired pressure.

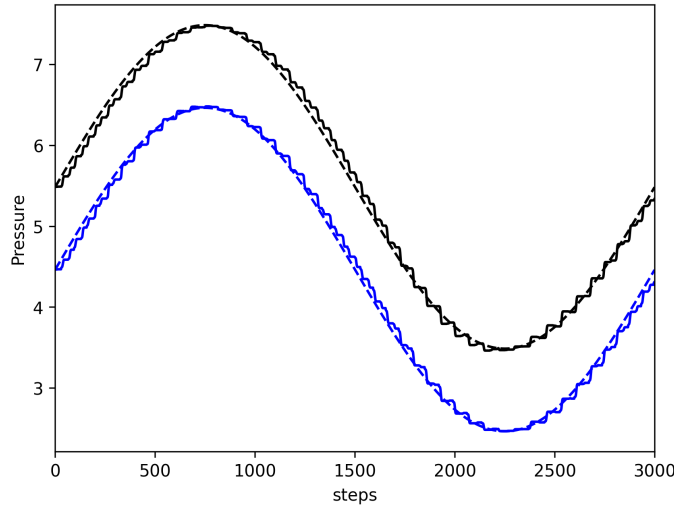


Figure 15: output pressure in solid line and desired pressure in dashed line for the two output nodes in black and blue, for a sinusodal desired pressure

For the constant pressure there are 58 chemical releases with an average interval of 51.2 steps between releases within the 3000 steps. The interval between releases in figure 16 starts of around 45 before settling into an oscillation between 49 and 56 caused by the flipping of the source and sink node.

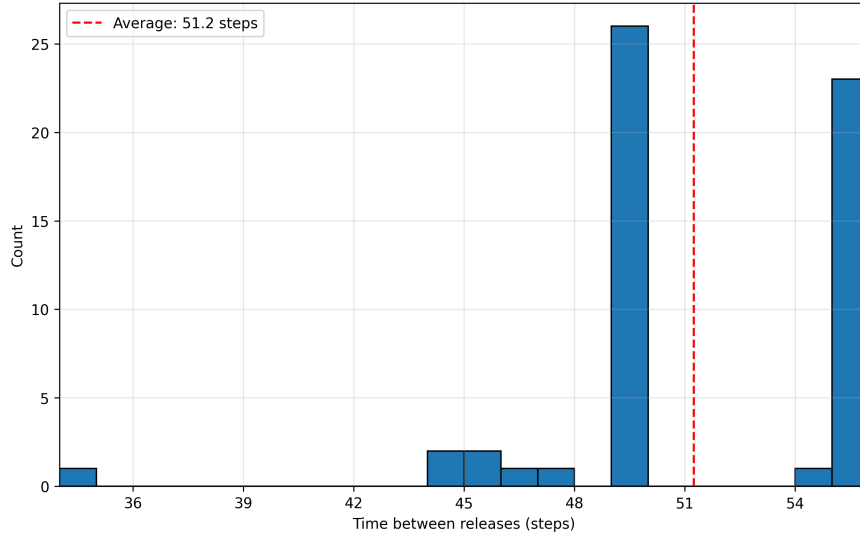


Figure 16: chemical release interval for linear desired pressure

for the sinusoidal pressure there are 87 releases with an average interval of 34.4 steps. Here the interval in figure 17 starts of oscitation between 29 and 35 and starts to drift because of the effects of the erosion.

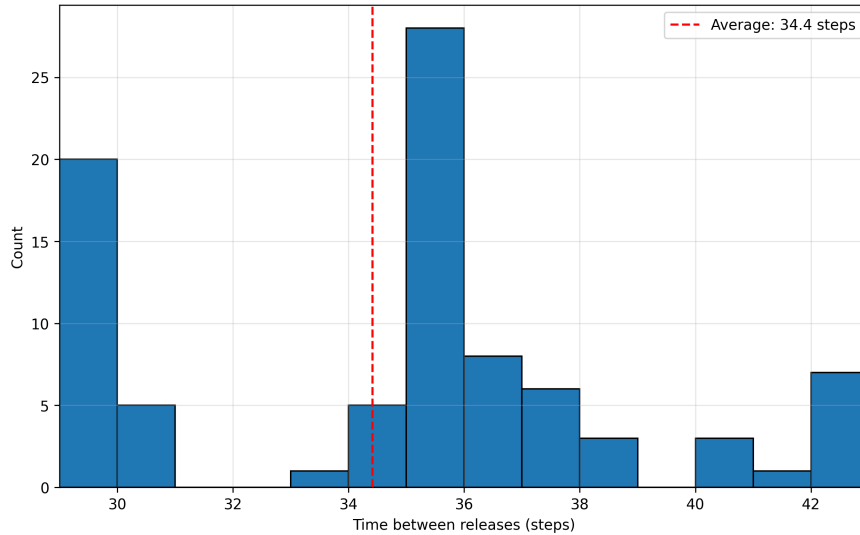


Figure 17: chemical release interval for sinusoidal desired pressure

For the linear desired pressure there are 90 chemical releases with an average interval of 33.1 steps. The interval first oscillate between 39 and 35 before settling on only 35 for both configuration of the boundary pressure.

3.3 machine learning

in this section we present the results of applying the chemical advection model as described in 2.3 on the iris dataset as described in 2.4

We start of with the starting network and flow of figure 18. The network has a size $N_x = N_y = 12$ there $N_x \cdot N_y$ nodes generated on the range $1 < x < N_x - 1$ and $0 < y < N_y$. There is a pressure drop between 4 pairs of output nodes on the left and right side of the network on $x = 0, N_x$. Direct edges between input nodes have been removed to prevent direct flow between input nodes. There are 3 output nodes in the middle of the network.

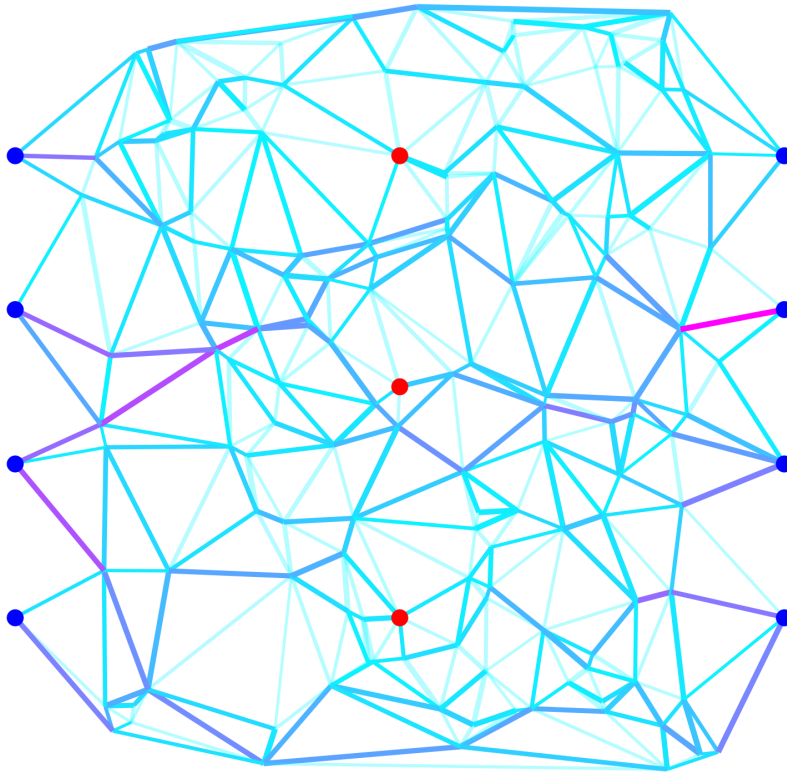


Figure 18: starting flow of the network for testing iris dataset of size $N_x = N_y = 12$. In the middle of the network 3 output nodes are added indicated in red at position $x = 0.5 \cdot N_x$ and $y = \frac{1}{4} \cdot N_y, \frac{1}{2} \cdot N_y$ and $\frac{3}{4} \cdot N_y$. There are 4 pairs of input nodes consisting of sink and source node indicated in blue located on $x = 0, N_x$ and $y = \frac{1}{4} \cdot N_y, \frac{1}{2} \cdot N_y$ and $\frac{3}{4} \cdot N_y$

The different types of iris form the dataset have been Labelled 0, 1 and 2 and coupled to a output node ordered from bottom to top. After training the model using the method of 2.4 we test the model on both the training data and the testing data . In table 1 we have the results for each entry for a random seed of 5. The training data results in an accuracy of 62.67% and the testing data results in an accuracy of 61.33%. This is s better than pure guessing which would result in an accuracy of 33.3%. We notice that the model has strong preference for choosing 0 and chooses 0 54% of the time for the test data and 54% for the training data. The model correctly predicts 0 in all cases.

4 Conclusions, discussion, and outlook

In this bachelor thesis we demonstrated that a flow network and simple physical inspired edge width adaption rules is capable of solving problems. firstly we simulated a boundary flux at train station nodes imbedded in a larger network. Together with a erosion law based on local flow and a clogging term based on local conductance. We showed can find a network connecting the Utrecht train stations and shows a large amount of similarities to the actual train network as seen in 5. We showed that changing the erosion and clogging paraments γ and β can change the topology of the network and result in a network with more edges 6. Directly connecting the station nodes without imbedding it in a larger network leads to similar network's with savings in computation cost: 8 and 10.

The network does not exactly match the real network network because of varies reasons. Firstly the Utrecht train region is not an isolated system and is the central hub in the train network of the Netherlands. where in our simulation it was treated as a isolated system. The simulation does not take numerous effects like rush hour into account. The construction of the train network isn't pure based on efficiency but on political, economical en geographical factors so simulating an accurate network will not converge to the actual network.

Secondly we introduced a chemical advection model where a chemical is released at output nodes based on the error to a desired output pressure. We showed that a erosion law based on the chemical flux convergence towards a desired constant output pressure 12. The model is able to follow a variable linear 14 and sinusoidal desired output pressure 15.

In this model we make the approximation that the chemical only jumps form node to node by advection each time step. Here we neglect the effects of the chemical concentration in the edges and diffusion effects.

lastly we applied the chemical advection model to the iris data set consisting of 150 entries of 4 inputs (iris characteristics) and 3 outputs (classes of iris) with the aim of classifying the right iris. For a specific random seed the model got an accuracy of 61.33 % 1 which is more accurate than random guessing which would result in a accuracy of 33.33 %. The specify accuracy depends on the random seeds and varies between 40 % and 67 %. The model often seems to struggle with predicting one class.

The accuracy of the training data doesn't transcends 67 %. The baseline model performance is typically above 90% so we are not able to reach these types of performances. It is noteworthy that this accuracy was achieved without conventional algorithms such as backpropagation, gradient decent or any other global optimization. This learning was achieved only solely with local and physically inspired update laws of the edge width.

Future researcher could simulate all train stations in the Netherlands. This could be computationally feasible without embedding the stations in a larger network wile taking the geography of the country such as the ijselmeer. The chemical advection model could be improved by taking advection into account and discretizing the edges and chemical concentration. The model could also be tested on more datasets than only the iris dataset. Conventional machine learning algorithms could be used on the flow network to see if they get a higher accuracy. If they get a higher accuracy a comparison could be used to further advance the local physical algorithms

5 Laymen summary (nederlands)

Een samenvatting in het Nederlands voor mensen die geïnteresseerd zijn in wat ik de afgelopen tijd gedaan heb tijdens mijn onderzoek maar niet de achtergrond in natuurkunde en Engels hebben om de hele thesis te lezen.

Introductie

De afgelopen jaren hebben large language models (kunstmatige intelligentie) zoals Chat-GPT en enorme opmars gemaakt. Deze ontwikkelingen komen ook met een groeiend probleem, een vraag aan een chatbot als Chat-GPT kost veel meer energie dan een google zoekopdracht. Voorspellingen lopen uiteen maar voorzichtig schattingen voorspellen dat kunstmatige intelligentie in 2030 evenveel energie gebruikt als een land als Japan. Onze hersenen zijn in vergelijking veel zuiniger met energie en kunnen met de energie van een appel complexe problemen oplossen. Slime molds zijn eencellige organisme zonder zenuwen die een netwerk vormen. Slime molds zijn wel in staat om problemen op te lossen door bijvoorbeeld een netwerk te maken dat voedselbronnen verbindt of het oplossen van een doolhof. Daarom is het doel om tijdens deze thesis geïnspireerd door slime molds stroom netwerken te gebruiken om problemen op te lossen.

Stroom netwerk

Allereerst wat is een stroom netwerk. Een stroom netwerk is een netwerk van punten verbonden met verbindingen waar een vloeistof doorheen stroomt enigszins vergelijkbaar met het netwerk van een rivierdelta. Om deze netwerken zo efficiënt mogelijk te simuleren maken we gebruik van lineaire algebra. Dit zijn reken operaties met matrices wat tabel getallen zijn geordend in rijen en kolommen; computers zijn geoptimaliseerd om hier zo snel mogelijk mee te rekenen.

Utrecht trein netwerk

We maken een vierkant stroom netwerken waar we de treinstations uit de provincie Utrecht aan toe voegen. Elke tijdstap laten we “water” in een station lopen en “water” uit een ander station lopen. Hierdoor stroomt het water van het ene station naar het andere station door het netwerk heen. Afhankelijk van hoeveel stroming er in een specifieke verbinding is laten we de grote van de verbinding eroderen (groeien) of versmallen opnieuw vergelijkbaar met een rivierdelta. Elke tijdstap kiezen we 2 nieuwe station en na verlopen van tijd ontstaat er een specifiek pad dat alle stations met elkaar verbindt. De link in de appendix heeft een animatie van dit proces. Het algoritme gebaseerd op fysieke wetten is dus in staat om een probleem op te lossen en het gevonden netwerk vertoont veel overeenkomsten het daadwerkelijke netwerk.

Signaalstof

Hierna hebben we een signaalstof toegevoegd aan de simulatie die zich met de stroming mee door het netwerk verplaatst. We kiezen 2 uitvoer punten in het netwerk waar we een specifieke uitvoer druk willen. Afhankelijk van of de druk hoger of lager is dan de gewenste uitvoer druk voegen we een + of – signaalstof toe aan het stroom netwerk. Als het element door het netwerk heen stroomt maakt het de verbindingen groter of kleiner afhankelijk van of het de + of - signaalstof is. In de appendix is een animatie van hoe de signaalstof door het netwerk heen stroomt. Hierdoor convergeert de uitvoer druk naar de gewenste uitvoer druk. Dit is vergelijkbaar met een slime mold die een hormoon vrij laat dat zijn netwerk her organiseert om te reageren op de omgeving.

Iris bloemen

We hebben dit signaalstof model toegepast op een iris bloem dataset. Hier zijn 3 verschillende type iris bloemen en 4 verschillende kenmerken van elke iris (blad lengte en breedte). Het doel is om aan de hand van de kenmerken te voorspellen welke iris het is. We gebruiken een stroom netwerk met 4 invoer punten die corresponderen met de kenmerken en 3 uitvoer die corresponderen met de iris type. Tijdens de training fase krijgt het model de 4 kenmerken de correcte iris. Met de signaalstof wordt de druk van een uitvoer punt verhoogt dat correspondeert met de juiste iris. Tijdens de test fase krijgt het model alleen de 4 kenmerken maar niet de juiste iris. Er wordt gekeken welk uitvoer punt voor de 4 kenmerken de hoogste druk heeft en of dit overeenkomt met het uitvoer punt van de juiste iris. Het model weet in meer dan 60% van de gevallen de juiste iris te kiezen wat ruim beter is dan gokken wat een nauwkeurigheid van 33 % zou hebben.

Conclusie

Ons stroom netwerk dat zich aanpast door natuurkundige wetten is in staat om een netwerk te maken dat de stations in Utrecht verbindt. Een stroom netwerk met een signaalstof dat het netwerk aanpast als het door het netwerk heen stroomt is in staat om met 60 % nauwkeurigheid de juiste iris te voorspellen van de iris dataset. Deze huidige modellen zijn nog niet indrukwekkend in vergelijking met Chat-GPT anderen computer algoritme, maar zijn een interessant alternatief dat basis kan vormen voor een alternatief toekomstige kunstmatige intelligentie en computer architectuur gebaseerd op de natuur.

References

- [1] A. Tero, S. Takagi, T. Saigusa, K. Ito, D. P. Bebber, M. D. Fricker, K. Yumiki, R. Kobayashi, and T. Nakagaki, *Science* **327**, 439 (2010), URL <https://doi.org/10.1126/science.1177894>.
- [2] UCI Machine Learning Repository, *Iris Data Set*, <https://archive.ics.uci.edu/ml/datasets/iris>.
- [3] International Energy Agency, Tech. Rep., International Energy Agency, Paris (2025), licence: CC BY 4.0, URL <https://www.iea.org/reports/energy-and-ai>.
- [4] J. You, *How much energy does chatgpt use?* (2025), published on Epoch AI, February 7, 2025, URL <https://epoch.ai/gradient-updates/how-much-energy-does-chatgpt-use>.
- [5] V. Balasubramanian, *Proceedings of the National Academy of Sciences of the United States of America* **118**, e2107022118 (2021), URL <https://doi.org/10.1073/pnas.2107022118>.
- [6] T. Nakagaki, H. Yamada, and Ágota Tóth, *Nature* **407**, 470 (2000), URL <https://doi.org/10.1038/35035159>.
- [7] M. Kramar and K. Alim, *Proc. Natl. Acad. Sci. U.S.A.* **118**, e2007815118 (2021), URL <https://doi.org/10.1073/pnas.2007815118>.
- [8] F. S. Gharehchopogh, A. Ucan, T. Ibriki, B. Arasteh, and G. Isik, *Archives of Computational Methods in Engineering* **30**, 2683 (2023), URL <https://doi.org/10.1007/s11831-023-09883-3>.
- [9] A. Zareei, D. Pan, and A. Amir, *Physical Review Letters* **128**, 234501 (2022), URL <https://doi.org/10.1103/PhysRevLett.128.234501>.
- [10] Rijden de Treinen, *Open data: Railway stations in the netherlands* (2023), URL <https://www.rijdendetreinen.nl/en/open-data/stations>.
- [11] Nederlandse Spoorwegen, *Gedetailleerde informatie*, <https://dashboards.ns.nl/hrncis> (2025), nS Dashboard. Accessed May 12, 2025.
- [12] V. Rao Aniseti, B. Scellier, and J. M. Schwarz, *Phys. Rev. Research* **5**, 023024 (2023), URL <https://doi.org/10.1103/PhysRevResearch.5.023024>.
- [13] Provincie Utrecht, *Spooragenda utrecht 2023* (2023), URL <https://www.provincialestatenutrecht.nl/documenten/PU23-054-spooragenda-2023-TG-2.pdf>.

A Appendix

The code used in this thesis together with the referenced animations and the images can be found on: [github](#)

B Result table

Training Data	Training Predicted	Testing Data	Testing Predicted
2	1	1	0
1	0	0	0
2	0	0	0
2	1	1	0
1	0	0	0
0	0	1	0
1	0	2	1
0	0	0	0
0	0	0	0
0	0	0	0
2	2	1	0
1	0	2	2
1	2	1	1
1	0	1	0
2	1	2	1
2	1	1	0
2	1	2	2
0	0	1	2
2	2	0	0
2	1	2	2
0	0	2	2
0	0	1	1
0	0	0	0
1	0	1	1
0	0	2	1
1	0	2	1
1	0	2	0
2	1	2	2
0	0	0	0
1	0	0	0
2	2	0	0
2	2	1	0
2	1	1	1
2	2	2	1
1	0	2	2
0	0	1	0

Training Data	Training Predicted	Testing Data	Testing Predicted
1	1	1	0
1	1	2	1
2	2	1	1
2	2	2	1
2	1	1	1
0	0	1	0
1	1	0	0
2	1	2	2
1	0	0	0
2	1	0	0
1	0	1	1
0	0	2	0
1	0	0	0
0	0	0	0
1	0	0	0
0	0	1	1
0	0	0	0
1	1	2	1
1	1	1	0
0	0	0	0
1	1	2	1
2	2	2	1
1	1	2	2
0	0	1	1
2	2	1	2
2	2	0	2
2	2	2	0
2	2	0	1
0	0	2	0
0	0	0	0
2	2	1	1
0	0	0	0
0	0	1	1
1	0	0	0
0	1	1	0
1	0	0	0
0	1	2	2
1	0	2	2
0	0	2	2
1	0	1	0

Training Data	Training Predicted	Testing Data	Testing Predicted
---------------	--------------------	--------------	-------------------

Table 1: Table of the data from the iris dataset where iris have been labeled 0, 1 and 2, split up into 75 training entries and 75 testing entries together with the prediction of the model for both the training and testing data. Test accuracy: 61.33%, Training accuracy: 62.67% for a random seed of 5.
